TRABAJO FINAL VISIÓN POR ORDENADOR

TIC - TAC - TOE

Blanca López-Jorrín Pérez

Victoria Guillén de la Torre

ÍNDICE

1. Introducción....3
1.1 Funcionamiento y reglas del juego....3
2. Calibración de la cámara....3
3. Diagrama de bloques del sistema....4
4. Sistema de Seguridad....4
5. Secuencia de transformación de la imagen....4
6. Sistema propuesto....5
6.1 Word Bag....5
6.2 Board....5
6.3 Find Quadrant....6
6.4 Computer player....6
6.5 Game Manager....6
7. Resultados....7

1. Introducción

En este proyecto final para la asignatura de' Visión por ordenador' se ha diseñado y desarrollado una versión innovadora del clásico juego *Tres en Raya*, en la que los jugadores pueden interactuar con el juego utilizando gestos realizados en el aire. Emplearán el algo de color rojo para indicar que quieren marcar una figura y el programa detectará no solo la figura si no también su posición.

El sistema se implementa utilizando una Raspberry Pi junto con una cámara y el entorno de Visual Studio.

1.1 Funcionamiento y reglas del juego

Sistema de seguridad: Para poder acceder al juego es necesario superar la seguridad del sistema. Más adelante se explicará cómo.

Selección del Modo de Juego: Al iniciar el juego, se presenta un menú donde puedes elegir entre jugar contra la computadora ("Modo Individual") o jugar con otro jugador ("Modo Multijugador").

Inicio del Juego: Tras seleccionar el modo, comienza una cuenta regresiva de 3 segundos, durante la cual el juego se prepara para el inicio.

Desarrollo del Juego:

- El jugador realiza su movimiento colocando su pieza en el cuadrante del tablero que elija. Si estás jugando contra la computadora, el ordenador realiza su movimiento de forma automática
- El jugador debe colocar la pieza en el espacio disponible, indicado en la pantalla. El sistema detecta la ubicación donde el jugador está "dibujando" en el aire con el color rojo para colocar la pieza.

Condición de Fin de Juego: El juego se detiene cuando uno de los jugadores logra una fila de tres símbolos consecutivos o cuando el tablero se llena sin que nadie haya ganado, lo que resultará en un empate.

2. Calibración de la cámara

Comenzamos el proyecto con la calibración de la cámara. Proceso que se hace una sola vez y offline. Para ello contamos con dos ficheros de python, *'save_photos.py' y 'calibration.py'*.

El primer fichero captura imágenes en tiempo real desde una cámara Raspberry Pi y las guarda en una carpeta llamada *'calibration_photos'*. Las imágenes que fueron seleccionadas las empleamos para la calibración de la cámara. El segundo fichero utiliza dichas imágenes para calcular los parámetros de calibración de la cámara, como la matriz intrínseca, los coeficientes de distorsión y la posición relativa al tablero. Los resultados se guardan en un archivo '.npz'.

Aunque estos resultados podrían emplearse para corregir dinámicamente la distorsión de la cámara durante la ejecución del juego, observamos que la distorsión era mínima y las diferencias entre aplicar o no la corrección resultaban despreciables. Por esta razón, decidimos no implementar dicha corrección, ya que el overhead adicional no justificaba su beneficio. De cualquier modo, se puede incluir dicha corrección fácilmente al inicio del fichero de juego.

3. Diagrama de bloques del sistema

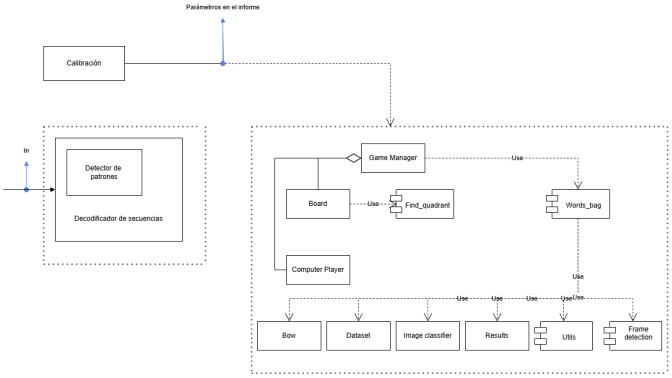

Figura 1: Diagrama de bloques

4. Sistema de Seguridad

Este sistema de seguridad permite la autenticación visual del usuario mediante una secuencia específica de colores y formas. El proceso comienza con una contraseña compuesta por tres pasos, en los cuales el usuario debe identificar un color y una forma en el entorno. Los colores incluyen rojo, verde y azul, y las formas son un cuadrado, un círculo y un pentágono.

Para detectar los colores, se utiliza el espacio de color HSV, lo que permite una mayor precisión en la identificación de tonos específicos. La detección de formas se realiza mediante la aproximación de contornos, verificando si la figura tiene el número de vértices correspondientes y un área dentro de un umbral determinado.

En cada paso, si el usuario acierta el color y la forma correspondientes, se avanza al siguiente paso. Los círculos en la pantalla indican visualmente el progreso del usuario, siendo vacíos al comienzo y llenándose conforme se avanza. Al completar todos los pasos correctamente, el sistema desactiva el modo de seguridad, otorgando acceso

La contraseña en nuestro caso ha sido la que secuencia que se ve en la Figura 2:

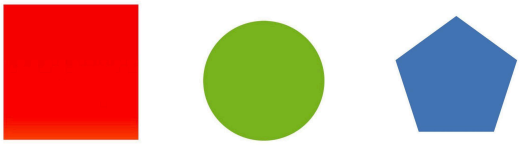

Figura 2: Contraseña del sistema de seguridad

5. Secuencia de transformación de la imagen

En este apartado explicaremos las diferentes transformaciones que se le realizan a los frames para así lograr analizarlos. Como a lo largo del trabajo se aplican un gran número de transformaciones, hemos querido resumir el procedimiento que seguimos según estos paso:

Recepción de la imagen: Se captura o recibe una imagen para procesar.

Preprocesamiento: Transformaciones iniciales como convertir el espacio de color o reducir el ruido.

Extracción de características: Se analiza la imagen para identificar atributos específicos, como colores o formas geométricas.

Validación: Se compara la información obtenida contra criterios predefinidos para determinar si se cumplen las condiciones esperadas.

Visualización: Se generan elementos gráficos en la imagen para comunicar el estado del sistema o los resultados.

Salida: Se devuelve la imagen procesada o los resultados del análisis para su uso posterior.

6. Sistema propuesto

A continuación, se desglosa el comportamiento de cada módulo individualmente, detallando su propósito y cómo contribuyen al funcionamiento general del sistema. Al finalizar este apartado, se espera ofrecer una visión clara y estructurada del funcionamiento global del sistema así como de sus interacciones.

6.1 Word Bag

El sistema utiliza un modelo de "Word Bag" para clasificar cruces y círculos a lo largo del juego. Los pasos principales son:

- Extracción de Descriptores: Se emplea SIFT (cv2.SIFT_create) para detectar características visuales en las imágenes del conjunto de entrenamiento.
- Creación del Vocabulario Visual: Los descriptores extraídos son agrupados mediante K-means en un vocabulario de palabras visuales, almacenado en un archivo para reutilización.
- Clasificación: El vocabulario se usa para entrenar un clasificador (ImageClassifier), que posteriormente identifica patrones visuales en nuevas imágenes.

En un primer momento, entrenamos el modelo utilizando imágenes obtenidas de Internet. Sin embargo, su rendimiento no fue el esperado, ya que no lograba clasificar correctamente los frames del juego. Por esta razón, decidimos generar imágenes propias, producidas de la misma forma en que se generarían durante la ejecución del juego.

Después de reentrenar el modelo con estas nuevas imágenes y validarlo, alcanzamos un rendimiento del 100%. Por ello, almacenamos el modelo en memoria para que pueda ser llamado fácilmente durante el juego, reduciendo significativamente el tiempo de procesamiento.

Además, se implementó un parámetro adicional que permite descartar imágenes que no correspondan a ninguna figura válida, asegurando una clasificación más precisa y robusta.

6.2 Board

La clase *'Board'* representa el tablero del juego de tres en raya y gestiona tanto su estructura como la interacción visual con los jugadores. Incluye las siguientes funcionalidades clave:

- *create_empty_grid():* Crea una matriz vacía que representa las casillas del tablero inicial sin movimientos registrados.
- *create_greed_coor():* Genera las coordenadas de las casillas del tablero para definir las posiciones de cada celda.
- *add_grid_coor():* Simula un movimiento de un jugador en la matriz simulada
- *draw_board():* Dibuja visualmente la situación del tablero sobre un frame
- *detect_board():* Dado un frame detecta las posiciones del tablero en px, solo se hace una vez en la partida .
- *save_move():* Almacena en coordenadas de la imagen un movimiento realizado, de forma que posteriormente se pueda visualizar con draw_board
- *detect_color():* identifica si está presente el color rojo dentro de un fotograma. Convierte el fotograma a HSV, genera máscaras para los colores definidos, y elimina el ruido. Luego, detecta contornos en la máscara combinada, selecciona el más grande, y calcula su centroide.
- *draw_trajectory():* dibuja sobre el frame la trayectoria que ha seguido el color rojo a lo largo de varios frames
- *get_trajectory():* dibuja sobre un fondo negro la misma trayectoria explicada anteriormente, con el objetivo de que sea más facil la identificación de la forma resultante por la bolsa de palabras.

En este apartado nos gustaría explicar un poco el proceso que hemos tenido hasta llegar a este diseño. Nuestra idea inicial fue detectar un tablero sin rayas extendiendo las líneas detectadas para formar su contorno. Sin embargo, este enfoque resultó poco práctico, ya que el tablero cambiaba en cada fotograma, lo que generaba inconsistencias. Además, detectar movimientos al colocar las piezas se volvía complicado con este método, por lo que decidimos descartar

6.3 Find Quadrant

Este módulo tiene como objetivo identificar el cuadrante del tablero en el que se encuentra el mayor número de puntos blancos, utilizando imágenes y análisis de esquinas. Primero, el módulo emplea el algoritmo visto en clase Shi-Tomasi para detectar las esquinas del tablero en una imagen y las ordena según sus posiciones relativas. A continuación, transforma la imagen en una binaria (blanco y negro), donde se cuenta el número de píxeles blancos dentro de cada uno de los nueve cuadrantes del tablero. El cuadrante con la mayor cantidad de puntos blancos se selecciona como el área de interés.

A través de esta metodología, dados el tablero y una imagen con una figura (es aquí donde empleamos el return del método '*get_trejectory*()' explicado en el apartado anterior) somos capaces de saber en qué cuadrante se ha dibujado dicha figura.

6.4 Computer player

Este módulo define la clase *ComputerPlayer*, que representa a un jugador controlado por la computadora en el juego. La clase incluye métodos para evaluar posibles movimientos, verificar el ganador o empate, y realizar el movimiento más óptimo utilizando el algoritmo Minimax. El algoritmo Minimax evalúa recursivamente todos los posibles movimientos en el tablero, asignando una puntuación a cada uno para maximizar la ganancia del jugador de la computadora ('O') y minimizar la del jugador contrario ('X'). El método '*computer_move()*' retorna el movimiento óptimo basado en el análisis realizado por Minimax.

6.5 Game Manager

La clase '*Game Manager*' es el corazón del juego y maneja la lógica de los diferentes modos y la interacción entre los jugadores. Podemos agrupar sus funcionalidades en grupos:

1. Análisis de la situación *('check_winner()', 'check_draw()', 'check_situation()'*):

Estas funciones determinan el estado del juego, verificando si hay un ganador, si el juego está empatado o si el juego debe continuar

2. Dibujo del menú ('*draw_menu()', 'draw_count_down()'*):

Muestran el menú principal y la cuenta atrás del inicio del juego sobre el video.

3. Control del juego ('*play_player()', 'play_computer()'*):

play_computer(): esta función hace uso del módulo de GamePlayer. Al ser llamada devuelve el siguiente movimiento del ordenador siguiendo el algoritmo Minimax.

'play_player(): esta función es más difícil de comprender. Pues como se explicó antes, el jugador para indicar la posición donde quiere colocar la pieza, deberá dibujarla en el aire en el cuadrante deseado y sosteniendo algo de color rojo. Es así que debemos de tener en cuenta, no solo el frame actual sino los anteriores. En la Figura 2 quisimos mostrar de manera esquemática las diferente acciones que hará en game manager en función de lo que haya pasado. Mientras el jugador no haya dibujado completamente su figura, se almacenarán las posiciones donde haya pasado el objeto rojo y se dibujará dicha trayectoria, por lo tanto no se cambiará de turno. Una vez haya acabado, se analizará la forma de dicha trayectoria y se procederá a realizar el movimiento.

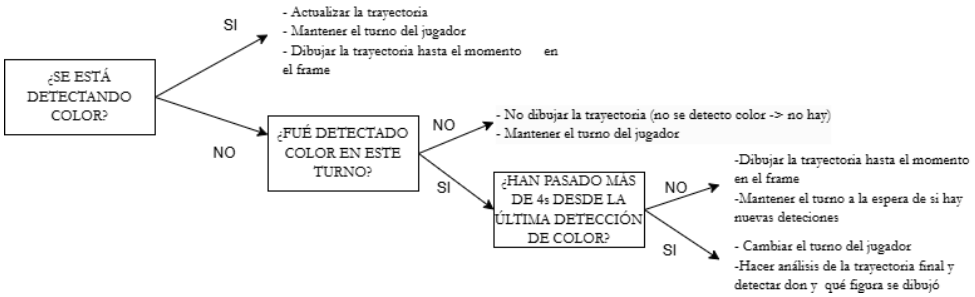

Figura 2: Esquema del funcionamiento de *'play_player*()'

4. Bucle principal ('*start_game*()')

La función gestiona el flujo principal del juego, comenzando con la selección del modo de juego (contra la computadora o multijugador). Primero, muestra un menú donde el jugador puede elegir entre jugar contra la computadora o en modo multijugador. Luego, inicia una cuenta regresiva antes de comenzar el juego. En el modo de juego, el turno alterna entre el jugador y la computadora, o entre dos jugadores,dependiendo del modo, permitiendo a cada uno realizar movimientos en el tablero detectando su posición a través de la cámara. El juego continúa hasta que se detecta un ganador o un empate, momento en el cual muestra el resultado y cierra el juego.

7. Resultados

GitHub link

https://github.com/blancalopezjorrin/ColorTicTacToe

Against the computer video

https://drive.google.com/file/d/1EQHBKw5-vKKY6CfHNfUzvgNZEPN6fDSg/view?usp=drive_link

Multiplayer video

https://drive.google.com/file/d/1F3CvCrxbhtlzRJo2wbbct-XG1stvMTsg/view?usp=sharing

Security Control Video

https://drive.google.com/file/d/1_4BUyIPK_l6u17rTTWgpb9dbQmTkk0V6/view?usp=sharing